Algorithms for Solving High Dimensional PDEs: From Nonlinear Monte Carlo to Machine Learning

Weinan E[1,2], Jiequn Han[1], and Arnulf Jentzen[3]

[1]Department of Mathematics, Princeton University
[2]Program in Applied and Computational Mathematics, Princeton University
[3]Faculty of Mathematics and Computer Science, University of Münster

September 14, 2020

Abstract

In recent years, tremendous progress has been made on numerical algorithms for solving partial differential equations (PDEs) in a very high dimension, using ideas from either nonlinear (multilevel) Monte Carlo or deep learning. They are potentially free of the curse of dimensionality for many different applications and have been proven to be so in the case of some nonlinear Monte Carlo methods for nonlinear parabolic PDEs.

In this paper, we review these numerical and theoretical advances. In addition to algorithms based on stochastic reformulations of the original problem, such as the multilevel Picard iteration and the Deep BSDE method, we also discuss algorithms based on the more traditional Ritz, Galerkin, and least square formulations. We hope to demonstrate to the reader that studying PDEs as well as control and variational problems in very high dimensions might very well be among the most promising new directions in mathematics and scientific computing in the near future.

Contents

Machine Learning and Computational Mathematics

Weinan E [i]

Princeton University and Beijing Institute of Big Data Research

In memory of Professor Feng Kang (1920-1993)

Contents

Abstract

Neural network-based machine learning is capable of approximating functions in very high dimension with unprecedented efficiency and accuracy. This has opened up many exciting new possibilities, not just in traditional areas of artificial intelligence, but also in scientific computing and computational science. At the same time, machine learning has also acquired the reputation of being a set of "black box"

[i] weinan@math.princeton.edu

Solving high-dimensional partial differential equations using deep learning

Jiequn Han[a], Arnulf Jentzen[b], and Weinan E[a,c,d,1]

[a]Program in Applied and Computational Mathematics, Princeton University, Princeton, NJ 08544; [b]Seminar for Applied Mathematics, Department of Mathematics, ETH Zürich, 8092 Zürich, Switzerland; [c]Department of Mathematics, Princeton University, Princeton, NJ 08544; and [d]Beijing Institute of Big Data Research, Beijing 100871, China

Edited by George Papanicolaou, Stanford University, Stanford, CA, and approved June 19, 2018 (received for review November 7, 2017)

Developing algorithms for solving high-dimensional partial differential equations (PDEs) has been an exceedingly difficult task for a long time, due to the notoriously difficult problem known as the "curse of dimensionality." This paper introduces a deep learning-based approach that can handle general high-dimensional parabolic PDEs. To this end, the PDEs are reformulated using backward stochastic differential equations and the gradient of the unknown solution is approximated by neural networks, very much in the spirit of deep reinforcement learning with the gradient acting as the policy function. Numerical results on examples including the nonlinear Black–Scholes equation, the Hamilton–Jacobi–Bellman equation, and the Allen–Cahn equation suggest that the proposed algorithm is quite effective in high dimensions, in terms of both accuracy and cost. This opens up possibilities in economics, finance, operational research, and physics, by considering all participating agents, assets, resources, or particles together at the same time, instead of making ad hoc assumptions on their interrelationships.

partial differential equations | backward stochastic differential equations | high dimension | deep learning | Feynman–Kac

Partial differential equations (PDEs) are among the most ubiquitous tools used in modeling problems in nature. Some of the most important ones are naturally formulated as PDEs in high dimensions. Well-known examples include the following:

i) The Schrödinger equation in the quantum many-body problem. In this case the dimensionality of the PDE is roughly three times the number of electrons or quantum particles in the system.

ii) The nonlinear Black–Scholes equation for pricing financial derivatives, in which the dimensionality of the PDE is the number of underlying financial assets under consideration.

iii) The Hamilton–Jacobi–Bellman equation in dynamic programming. In a game theory setting with multiple agents, the dimensionality goes up linearly with the number of agents. Similarly, in a resource allocation problem, the dimensionality goes up linearly with the number of devices and resources.

As elegant as these PDE models are, their practical use has proved to be very limited due to the curse of dimensionality (1): The computational cost for solving them goes up exponentially with the dimensionality.

Another area where the curse of dimensionality has been an essential obstacle is machine learning and data analysis, where the complexity of nonlinear regression models, for example, goes up exponentially with the dimensionality. In both cases the essential problem we face is how to represent or approximate a nonlinear function in high dimensions. The traditional approach, by building functions using polynomials, piecewise polynomials, wavelets, or other basis functions, is bound to run into the curse of dimensionality problem.

In recent years a new class of techniques, the deep neural network model, has shown remarkable success in artificial intelligence (e.g., refs. 2–6). The neural network is an old idea but recent experience has shown that deep networks with many layers seem to do a surprisingly good job in modeling complicated datasets. In terms of representing functions, the neural network model is compositional: It uses compositions of simple functions to approximate complicated ones. In contrast, the approach of classical approximation theory is usually additive. Mathematically, there are universal approximation theorems stating that a single hidden-layer neural network can approximate a wide class of functions on compact subsets (see, e.g., survey in ref. 7 and the references therein), even though we still lack a theoretical framework for explaining the seemingly unreasonable effectiveness of multilayer neural networks, which are widely used nowadays. Despite this, the practical success of deep neural networks in artificial intelligence has been very astonishing and encourages applications to other problems where the curse of dimensionality has been a tormenting issue.

In this paper, we extend the power of deep neural networks to another dimension by developing a strategy for solving a large class of high-dimensional nonlinear PDEs using deep learning. The class of PDEs that we deal with is (nonlinear) parabolic PDEs. Special cases include the Black–Scholes equation and the Hamilton–Jacobi–Bellman equation. To do so, we make use of the reformulation of these PDEs as backward stochastic differential equations (BSDEs) (e.g., refs. 8 and 9) and approximate the gradient of the solution using deep neural networks. The methodology bears some resemblance to deep reinforcement learning with the BSDE playing the role of model-based reinforcement learning (or control theory models) and the gradient of the solution playing the role of policy function. Numerical

Significance

Partial differential equations (PDEs) are among the most ubiquitous tools used in modeling problems in nature. However, solving high-dimensional PDEs has been notoriously difficult due to the "curse of dimensionality." This paper introduces a practical algorithm for solving nonlinear PDEs in very high (hundreds and potentially thousands of) dimensions. Numerical results suggest that the proposed algorithm is quite effective for a wide variety of problems, in terms of both accuracy and speed. We believe that this opens up a host of possibilities in economics, finance, operational research, and physics, by considering all participating agents, assets, resources, or particles together at the same time, instead of making ad hoc assumptions on their interrelationships.

Author contributions: J.H., A.J., and W.E. designed research, performed research, and wrote the paper.

The authors declare no conflict of interest.

This article is a PNAS Direct Submission.

[1]To whom correspondence should be addressed. Email: weinan@math.princeton.edu.

Published online August 6, 2018.

A Quasi-Monte Carlo Data Structure for Smooth Kernel Evaluations

Moses Charikar* Stanford University

Michael Kapralov† EPFL

Erik Waingarten‡ University of Pennsylvania

January 8, 2024

Abstract

In the kernel density estimation (KDE) problem one is given a kernel $K(x, y)$ and a dataset P of points in a high dimensional Euclidean space, and must prepare a small space data structure that can quickly answer density queries: given a point q, output a $(1+\epsilon)$-approximation to $\mu := \frac{1}{|P|}\sum_{p\in P} K(p, q)$. The classical approach to KDE (and the more general problem of matrix vector multiplication for kernel matrices) is the celebrated fast multipole method of Greengard and Rokhlin [1983]. The fast multipole method combines a basic space partitioning approach with a multidimensional Taylor expansion, which yields a $\approx \log^d(n/\epsilon)$ query time (exponential in the dimension d). A recent line of work initiated by Charikar and Siminelakis [2017] achieved polynomial dependence on d via a combination of random sampling and randomized space partitioning, with Backurs et al. [2018] giving an efficient data structure with query time $\approx \text{poly}(\log(1/\mu))/\epsilon^2$ for smooth kernels.

Quadratic dependence on ϵ, inherent to the sampling (i.e., Monte Carlo) methods above, is prohibitively expensive for small ϵ. This is a classical issue addressed by quasi-Monte Carlo methods in numerical analysis. The high level idea in quasi-Monte Carlo methods is to replace random sampling with a discrepancy based approach – an idea recently applied to coresets for KDE by Phillips and Tai [2020]. The work of Phillips and Tai gives a space efficient data structure with query complexity $\approx 1/(\epsilon\mu)$. This is polynomially better in $1/\epsilon$, but exponentially worse in $1/\mu$. In this work we show how to get the best of both worlds: we give a data structure with $\approx \text{poly}(\log(1/\mu))/\epsilon$ query time for smooth kernel KDE. Our main insight is a new way to combine discrepancy theory with randomized space partitioning inspired by, but significantly more efficient than, that of the fast multipole methods. We hope that our techniques will find further applications to linear algebra for kernel matrices.

*Supported by a Simons Investigator award.

†Supported in part by ERC Starting Grant 759471.

‡Part of this work was done as a postdoc at Stanford University, supported by an NSF postdoctoral fellowship and Moses Charikar's Simons Investigator award.

Adaptive Multidimensional Integration Based on Rank-1 Lattices

Lluís Antoni Jiménez Rugama and Fred J. Hickernell

Abstract Quasi-Monte Carlo methods are used for numerically integrating multivariate functions. However, the error bounds for these methods typically rely on a priori knowledge of some semi-norm of the integrand, not on the sampled function values. In this article, we propose an error bound based on the discrete Fourier coefficients of the integrand. If these Fourier coefficients decay more quickly, the integrand has less fine scale structure, and the accuracy is higher. We focus on rank-1 lattices because they are a commonly used quasi-Monte Carlo design and because their algebraic structure facilitates an error analysis based on a Fourier decomposition of the integrand. This leads to a guaranteed adaptive cubature algorithm with computational cost $\mathscr{O}(mb^m)$, where b is some fixed prime number and b^m is the number of data points.

1 Introduction

Quasi-Monte Carlo (QMC) methods use equally weighted sums of integrand values at carefully chosen nodes to approximate multidimensional integrals over the unit cube,

$$\frac{1}{n}\sum_{i=0}^{n-1} f(\boldsymbol{z}_i) \approx \int_{[0,1)^d} f(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}.$$

Integrals over more general domains may often be accommodated by a transformation of the integration variable. QMC methods are widely used because they do not suffer from a *curse of dimensionality*. The existence of QMC methods with dimension-independent error convergence rates is discussed in [11, Ch. 10–12]. See [3] for a recent review.

Lluís Antoni · Fred J. Hickernell
Department of Applied Mathematics, Illinois Institute of Technology, 10 W. 32$^{\text{nd}}$ Street, E1-208, Chicago, IL 60616, USA e-mail: ljimene1@hawk.iit.edu, e-mail: hickernell@iit.edu

An Extreme Learning Machine-Based Method for Computational PDEs in Higher Dimensions

Yiran Wang, Suchuan Dong*

Center for Computational & Applied Mathematics
Department of Mathematics
Purdue University, USA

(September 13, 2023)

Abstract

We present two effective methods for solving high-dimensional partial differential equations (PDE) based on randomized neural networks. Motivated by the universal approximation property of this type of networks, both methods extend the extreme learning machine (ELM) approach from low to high dimensions. With the first method the unknown solution field in d dimensions is represented by a randomized feed-forward neural network, in which the hidden-layer parameters are randomly assigned and fixed while the output-layer parameters are trained. The PDE and the boundary/initial conditions, as well as the continuity conditions (for the local variant of the method), are enforced on a set of random interior/boundary collocation points. The resultant linear or nonlinear algebraic system, through its least squares solution, provides the trained values for the network parameters. With the second method the high-dimensional PDE problem is reformulated through a constrained expression based on an Approximate variant of the Theory of Functional Connections (A-TFC), which avoids the exponential growth in the number of terms of TFC as the dimension increases. The free field function in the A-TFC constrained expression is represented by a randomized neural network and is trained by a procedure analogous to the first method. We present ample numerical simulations for a number of high-dimensional linear/nonlinear stationary/dynamic PDEs to demonstrate their performance. These methods can produce accurate solutions to high-dimensional PDEs, in particular with their errors reaching levels not far from the machine accuracy for relatively lower dimensions. Compared with the physics-informed neural network (PINN) method, the current method is both cost-effective and more accurate for high-dimensional PDEs.

Key words: high-dimensional PDE, extreme learning machine, randomized neural network, deep neural network, scientific machine learning, deep learning

1 Introduction

This work concerns the numerical approximation of partial differential equations (PDEs) in higher dimensions (typically beyond three). Mathematical models describing natural and physical processes or phenomena are usually expressed in PDEs. In a number of fields and domains, including physics, biology and finance, the models are naturally formulated in terms of high-dimensional PDEs. Well-known examples include the Schrodinger equation for many-body problems in quantum mechanics, the Black-Scholes equation for the price evolution of financial derivatives, and the Hamilton-Jacobi-Bellman (HJB) equation in dynamic programming and game theory [30, 78]. Development of computational techniques for PDEs is a primary thrust in scientific computing. In low dimensions, traditional numerical methods such as the finite difference, finite element (FEM), finite volume, and spectral type methods (and their variants), which are typically grid- or mesh-based, have achieved a tremendous success and are routinely used in computational science

*Author of correspondence. Emails: wang2335@purdue.edu (Y. Wang), sdong@purdue.edu (S. Dong)

ANZIAM J. 53(2011), 1–37
doi:10.1017/S1446181112000077

QUASI-MONTE CARLO METHODS FOR HIGH-DIMENSIONAL INTEGRATION: THE STANDARD (WEIGHTED HILBERT SPACE) SETTING AND BEYOND

F. Y. KUO[1], CH. SCHWAB[2] and I. H. SLOAN[✉1,*]

(Received 30 December, 2011; revised 3 February, 2012)

Abstract

This paper is a contemporary review of quasi-Monte Carlo (QMC) methods, that is, equal-weight rules for the approximate evaluation of high-dimensional integrals over the unit cube $[0, 1]^s$. It first introduces the by-now standard setting of weighted Hilbert spaces of functions with square-integrable mixed first derivatives, and then indicates alternative settings, such as non-Hilbert spaces, that can sometimes be more suitable. Original contributions include the extension of the fast component-by-component (CBC) construction of lattice rules that achieve the optimal convergence order (a rate of almost $1/N$, where N is the number of points, independently of dimension) to so-called "product and order dependent" (POD) weights, as seen in some recent applications. Although the paper has a strong focus on lattice rules, the function space settings are applicable to all QMC methods. Furthermore, the error analysis and construction of lattice rules can be adapted to polynomial lattice rules from the family of digital nets.

2010 *Mathematics subject classification*: primary 65D30; secondary 65D32.

Keywords and phrases: quasi-Monte Carlo methods, QMC, high-dimensional integration, weighted spaces, reproducing kernel Hilbert spaces, Banach space settings, worst-case error, discrepancy, weighted Koksma–Hlawka inequality, lattice rules, low-discrepancy sequences, fast CBC construction, POD weights.

1. Introduction and some QMC basics

Quasi-Monte Carlo (QMC) methods are deterministic methods for high-dimensional integration which aim to outperform the classical Monte Carlo method. In the past fifteen years great progress has been made, often in the setting of "worst-case" errors in a "weighted" Hilbert space, as introduced originally by Sloan and Woźniakowski [53].

---

*This is a contribution to the series of invited papers by past ANZIAM medallists (Editorial, issue 52(1)). Ian Sloan was awarded the 1997 ANZIAM medal.

[1]School of Mathematics and Statistics, University of New South Wales, Sydney NSW 2052, Australia; e-mail: f.kuo@unsw.edu.au, i.sloan@unsw.edu.au.
[2]Seminar for Applied Mathematics, ETH Zürich, ETH Zentrum, HG G57.1, CH8092 Zürich, Switzerland; e-mail: christoph.schwab@sam.math.ethz.ch.

Journal of Computational Physics 538 (2025) 114160

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

Non-uniform random walk for adaptive sampling

Rouhan Wang, Dan Hu*

School of Mathematical Sciences, Institute of Natural Sciences, and MOE-LSC, Shanghai Jiao Tong University, Shanghai, 200240, China

A R T I C L E   I N F O

2000 MSC:
62M45
65K10
68T20
82B41

Keywords:
Deep learning
Physics-informed neural networks
Adaptive sampling
Random walk

A B S T R A C T

Physics-Informed Neural Networks (PINNs) have been widely used in solving partial differential equations. In order to accelerate the convergence and improve the numerical accuracy of solutions, especially for solutions with low regularity, adaptive sampling strategies have been incorporated into PINNs, which place more sample points in regions with large residual. We propose a Non-Uniform Random Walk process for Adaptive Sampling (Nurwas). In Nurwas, sample points undergo a non-uniform random walk, with step-sizes inversely proportional to the square root of the desired probability density. Since the desired distribution is usually given by the residual of the numerical solution, Nurwas is performed without an explicit expression of the probability density function and without additional computational cost. As a result, compared to previous adaptive sampling strategies, Nurwas simplifies the sampling process and avoids the difficulties and limitations introduced by explicitly expressing the probability density function.

1. Introduction

Partial differential equation (PDE) is an important tool for modeling real-world problems. Traditional numerical methods for solving PDE, such as the finite difference method, finite element method (FEM), and spectral methods, have achieved wide and significant accomplishments. Nevertheless, traditional numerical methods suffer from the curse of dimensionality for high dimensional PDE and the engineering complexity of irregular geometry. Deep neural networks (DNN) provide a new approach to solving PDE, which can potentially overcome the difficulties of traditional numerical methods [1–4].

As a widely used deep-learning method for PDE, Physics-Informed neural networks (PINNs) [1,5,6] reformulate a specific PDE as a functional optimization problem and use a DNN to approximate the solution. In order to train the parameters of the DNN, the integral is usually approximated by the Monte Carlo method with uniformly distributed sample points within the problem domain. The uniform random sampling strategy may be inefficient for a certain PDE when the solution has low regularities due to the large variance in the Monte Carlo approximation.

Efforts have been made to reduce the variance in the Monte Carlo approximation in PINNs. Adaptive sampling methods have been developed to accelerate the convergence of PINNs [6–11]. The Residual-based adaptive refinement method (RAR) [6] was proposed to augment the training set with new points having larger residual values. Later, the residual-based adaptive distribution (RAD) and Residual-based adaptive refinement with distribution (RAR-D) [7] methods introduced a combination of residual-induced distribution and uniform distribution for sampling. Inspired by the idea of importance sampling, the deep adaptive sampling method (DAS) [8] combines the training of two neural networks, PINNs and KRnet, by generating the training dataset with the proposed density through KRnet, which efficiently captures the singularities. The failure-informed adaptive sampling method (FI-PINNs) [9,10] approximates the failure probability of PINNs using Gaussian or subset simulation and adds new collocation points from the failure region to the training set. This method shows improvement in solving complex and high-dimensional problems. In order to reduce the complexity

* Corresponding author.
E-mail addresses: wangrh1127@sjtu.edu.cn (R. Wang), hudan80@sjtu.edu.cn (D. Hu).

https://doi.org/10.1016/j.jcp.2025.114160
Received 6 July 2024; Received in revised form 7 April 2025; Accepted 3 June 2025
Available online 13 June 2025

PDENNEval: A Comprehensive Evaluation of Neural Network Methods for Solving PDEs*

Ping Wei, **Menghan Liu**, **Jianhuan Cen**, **Ziyang Zhou**, **Liao Chen** and **Qingsong Zou**†
School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China
Guangdong Province Key Laboratory of Computational Science, Guangzhou, China
{weip7, liumh59, cenjh3, zhouzy36, chenliao}@mail2.sysu.edu.cn, mcszqs@mail.sysu.edu.cn

Abstract

The rapid development of neural network (NN) methods for solving partial differential equations (PDEs) has created an urgent need for evaluation and comparison of these methods. In this study, we propose PDENNEval, a comprehensive and systematic evaluation of 12 NN methods for PDEs. These methods are classified into function learning type and operator learning type based on their different mathematical foundations. The evaluation is implemented using a diverse dataset comprising 19 distinct PDE problems selected from various scientific fields such as fluid, materials, finance, and electromagnetic. Several evaluation results are reported, aiming to provide guidance for further research in this field. Our code and data are publicly available at https://github.com/Sysuzqs/PDENNEval.

1 Introduction

Owing to their exceptional approximation capabilities [Hornik, 1991; Barron, 1994] and multi-level feature extraction abilities [LeCun *et al.*, 2015], neural networks (NN) based methods have been widely applied and have achieved great success in various disciplines, including computer vision [Krizhevsky *et al.*, 2012; He *et al.*, 2016; Dosovitskiy *et al.*, 2020] and natural language processing ([Vaswani *et al.*, 2017; Devlin *et al.*, 2018; Brown *et al.*, 2020]). Recently, NN methods have excelled in solving partial differential equations (PDEs), yielding significant breakthroughs in this field, e.g. [E *et al.*, 2017; E and Yu, 2018; Raissi *et al.*, 2019; Li *et al.*, 2020; Lu *et al.*, 2021a]. Compared to traditional numerical PDEs methods like finite element methods, NN methods exhibit unique advantages. One notable advantage is that they are mesh-free, making them immune to the *curse of dimensionality*. In addition, NN methods can learn patterns and features from data which might be governed by multiple types of equations[Raissi and Karniadakis, 2018]. This capability enables a single solver to handle various types of equations, offering a more versatile approach for solving PDEs.

The widespread development of NN methods for solving PDEs generates a pressing need for the assessment and comparative analysis of these techniques. In 2021, Lu and his colleagues compare the Physics-Informed Neural Networks (PINN) to a standard finite element method across various problems, including Poisson equation, Burgers' equation, Volterra integral differential equation, Lorenz system and Diffusion-Reaction systems in [Lu *et al.*, 2021b]. In 2022, three benchmarks of NN methods for PDEs are published. First, Lu and his colleagues[Lu *et al.*, 2022] present the relative performance of Deep Operator Network (DeepONet) and Fourier Neural Operator (FNO) by using 16 different benchmarks covering Burgers' equation, Darcy problem, Advection equation, Compressible Euler equations, electro-convection problem and Navier-Stokes equation, etc. Secondly, a side-by-side analysis, called PDEArena, is presented in [Gupta and Brandstetter, 2022] to show the performance of NN methods FNO, ResNet, and U-Net on Shallow-Water equations and Navier-Stokes equations. Thirdly, a versatile benchmark suite[Takamoto *et al.*, 2022b], called PDEBench, is proposed to compare NN methods FNO, U-Net, PINN, using datasets derived from 11 hydromechanical field PDEs, including Advection, Navier-Stokes equations, etc. In 2023, a benchmark[Hao *et al.*, 2023], called PINNacle, is designed to assess the performance of 10 PINN variants, encompassed tasks derived from over 20 different PDEs spanning various domains, such as heat conduction, fluid dynamics, biology, and electromagnetic.

Previous research primarily benchmarks some popular methods such as DeepONet, FNO, U-Net, and PINN. However, a comprehensive evaluation of some other important NN methods is lacking. Additionally, different types of NN methods are often compared in the same setting without being classified and evaluated based on their distinct mathematical foundations. On the other hand, despite the selection of various types of PDE problems in previous work, some crucial equations, such as phase-field equations, have not been included in the evaluation tasks. Furthermore, challenging problems like PDEs with singular solutions have also been omitted. In summary, the development of the evaluation and comparison works for NN methods have not kept pace with the advancement of NN methods themselves.

*An extendede version of this paper(with the Appendix included) can be find in https://github.com/Sysuzqs/PDENNEval.

†Corresponding author.

Numer. Math. Theor. Meth. Appl.
doi: 10.4208/nmtma.OA-2020-0062

Published online 9 October 2020

Quasi-Monte Carlo Sampling for Solving Partial Differential Equations by Deep Neural Networks

Jingrun Chen[1], Rui Du[1,*], Panchi Li[2] and Liyao Lyu[3]

[1] *School of Mathematical Sciences and Mathematical Center for Interdisciplinary Research, Soochow University, Suzhou 215006, China*
[2] *School of Mathematical Sciences, Soochow University, Suzhou 215006, China*
[3] *School of Mathematical Sciences and CW Chu College, Soochow University, Suzhou 215006, China*

Received 14 April 2020; Accepted (in revised version) 7 September 2020

Abstract. Solving partial differential equations in high dimensions by deep neural networks has brought significant attentions in recent years. In many scenarios, the loss function is defined as an integral over a high-dimensional domain. Monte-Carlo method, together with a deep neural network, is used to overcome the curse of dimensionality, while classical methods fail. Often, a neural network outperforms classical numerical methods in terms of both accuracy and efficiency. In this paper, we propose to use quasi-Monte Carlo sampling, instead of Monte-Carlo method to approximate the loss function. To demonstrate the idea, we conduct numerical experiments in the framework of deep Ritz method. For the same accuracy requirement, it is observed that quasi-Monte Carlo sampling reduces the size of training data set by more than two orders of magnitude compared to that of Monte-Carlo method. Under some assumptions, we can prove that quasi-Monte Carlo sampling together with the deep neural network generates a convergent series with rate proportional to the approximation accuracy of quasi-Monte Carlo method for numerical integration. Numerically the fitted convergence rate is a bit smaller, but the proposed approach always outperforms Monte Carlo method.

AMS subject classifications: 11K50, 35J20, 65N99

Key words: Quasi-Monte Carlo sampling, deep Ritz method, loss function, convergence analysis.

1. Introduction

Deep neural networks (DNNs) have had great success in text classification, computer vision, natural language processing and other data-driven applications [13, 16,

*Corresponding author. *Email addresses:* jingrunchen@suda.edu.cn (J. Chen), durui@suda.edu.cn (R. Du), LiPanchi1994@163.com (P. Li), lylv@stu.suda.edu.cn (L. Lyu)

Neural Networks 176 (2024) 106369

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Tackling the curse of dimensionality with physics-informed neural networks

Zheyuan Hu [a,*], Khemraj Shukla [b], George Em Karniadakis [b], Kenji Kawaguchi [a]

[a] National University of Singapore, 21 Lower Kent Ridge Road, 119077, Singapore
[b] Division of Applied Mathematics, Brown University, 182 George Street, Providence, RI 02912, USA

ARTICLE INFO

Keywords:
Physics-informed neural networks
Curse of dimensionality

ABSTRACT

The curse-of-dimensionality taxes computational resources heavily with exponentially increasing computational cost as the dimension increases. This poses great challenges in solving high-dimensional partial differential equations (PDEs), as Richard E. Bellman first pointed out over 60 years ago. While there has been some recent success in solving numerical PDEs in high dimensions, such computations are prohibitively expensive, and true scaling of general nonlinear PDEs to high dimensions has never been achieved. We develop a new method of scaling up physics-informed neural networks (PINNs) to solve arbitrary high-dimensional PDEs. The new method, called Stochastic Dimension Gradient Descent (SDGD), decomposes a gradient of PDEs' and PINNs' residual into pieces corresponding to different dimensions and randomly samples a subset of these dimensional pieces in each iteration of training PINNs. We prove theoretically the convergence and other desired properties of the proposed method. We demonstrate in various diverse tests that the proposed method can solve many notoriously hard high-dimensional PDEs, including the Hamilton–Jacobi-Bellman (HJB) and the Schrödinger equations in tens of thousands of dimensions very fast on a single GPU using the PINNs mesh-free approach. Notably, we solve nonlinear PDEs with nontrivial, anisotropic, and inseparable solutions in less than one hour for 1000 dimensions and in 12 h for 100,000 dimensions on a single GPU using SDGD with PINNs. Since SDGD is a general training methodology of PINNs, it can be applied to any current and future variants of PINNs to scale them up for arbitrary high-dimensional PDEs.

1. Introduction

The curse-of-dimensionality (CoD) refers to the computational and memory challenges when dealing with high-dimensional problems that do not exist in low-dimensional settings. The term was first introduced in 1957 by Richard E. Bellman (Bellman, 1957; Hammer, 1962), who was working on dynamic programming, to describe the exponentially increasing costs due to high dimensions. Later, the concept was adopted in various areas of science, including numerical partial differential equations (PDEs), combinatorics, machine learning, probability, stochastic analysis, and data mining. In the context of numerically solving PDEs, an increase in the dimensionality of the PDE's independent variables tends to lead to a corresponding exponential rise in computational costs needed for obtaining an accurate solution. This poses a considerable challenge for all existing algorithms.

Physics-Informed Neural Networks (PINNs) are highly practical in solving PDE problems (Raissi, Perdikaris, & Karniadakis, 2019). Compared to other methods, PINNs can solve various PDEs in general form, enabling mesh-free solutions and handling complex geometries. Additionally, PINNs leverage the interpolation capability of neural networks to provide accurate predictions throughout the domain, unlike other methods for high-dimensional PDEs, which can only compute the value of PDEs at a single point or are restricted to certain PDE types (Beck, Becker, Cheridito, Jentzen, & Neufeld, 2021; Han, Jentzen, & E, 2018; Raissi, 2018). Due to neural networks' flexibility and expressiveness, in principle, the set of functions representable by PINNs have the ability to approximate high-dimensional PDE solutions. However, PINNs may also fail in tackling CoD due to insufficient memory and slow convergence when dealing with high-dimensional PDEs. For example, when the dimension of PDE is very high, even just using one collocation point can lead to an insufficient memory error due to the massive high-dimensional derivatives in the PDE. Unfortunately, the aforementioned high-dimensional and high-order cases often occur for very useful PDEs, such as the Hamilton–Jacobi-Bellman (HJB) equation in stochastic optimal control, the Fokker–Planck equation in stochastic analysis and high-dimensional probability, and the Black–Scholes equation in mathematical finance, etc. These PDE examples pose a grand challenge to the application of PINNs in effectively tackling real-world large-scale problems.

* Corresponding author.
E-mail addresses: e0792494@u.nus.edu (Z. Hu), khemraj_shukla@brown.edu (K. Shukla), george_karniadakis@brown.edu (G.E. Karniadakis), kenji@nus.edu.sg (K. Kawaguchi).

https://doi.org/10.1016/j.neunet.2024.106369
Received 13 November 2023; Received in revised form 3 March 2024; Accepted 5 May 2024
Available online 7 May 2024

VARIANCE-REDUCING COUPLINGS FOR RANDOM FEATURES

Isaac Reid[1], **Stratis Markou**[1], **Krzysztof Choromanski**[2,3], **Richard E. Turner**[1], **Adrian Weller**[1,4]
[1]University of Cambridge, [2]Google DeepMind, [3]Columbia, [4]Alan Turing Institute

ABSTRACT

Random features (RFs) are a popular technique to scale up kernel methods in machine learning, replacing exact kernel evaluations with stochastic Monte Carlo estimates. They underpin models as diverse as efficient transformers (by approximating attention) to sparse spectrum Gaussian processes (by approximating the covariance function). Efficiency can be further improved by speeding up the convergence of these estimates: a *variance reduction* problem. We tackle this through the unifying lens of *optimal transport*, finding couplings to improve RFs defined on both Euclidean and discrete input spaces. They enjoy theoretical guarantees and sometimes provide strong downstream gains, including for scalable approximate inference on graphs. We reach surprising conclusions about the benefits and limitations of variance reduction as a paradigm, showing that other properties of the coupling should be optimised for attention estimation in efficient transformers.

1 INTRODUCTION

Kernel methods are ubiquitous in machine learning (Canu and Smola, 2006; Smola and Schölkopf, 2002; Kontorovich et al., 2008; Campbell, 2002). Through the kernel trick, they provide a mathematically principled and elegant way to perform nonlinear inference using linear learning algorithms. The eponymous positive definite *kernel function* $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ measures the 'similarity' between two datapoints. The input domain $\mathcal{X}$ may be continuous, e.g. the set of vectors in $\mathbb{R}^d$, or discrete, e.g. the set of graph nodes or entire graphs.

Random features for kernel approximation. Though very effective on small datasets, kernel methods suffer from poor scalability. The need to materialise and invert the *Gram matrix* $\mathbf{K} := [k(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{i=1}^N$ leads to a time complexity cubic in the size of the dataset N. Substantial research has been dedicated to improving scalability by approximating this matrix, a prominent example being *random features* (RFs) (Rahimi and Recht, 2007; 2008; Avron et al., 2017b; Liu et al., 2022). These randomised mappings $\phi : \mathbb{R}^d \to \mathbb{R}^s$ construct low-dimensional or sparse feature vectors that satisfy

$$k(\boldsymbol{x}, \boldsymbol{y}) = \mathbb{E}\left(\phi(\boldsymbol{x})^\top \phi(\boldsymbol{y})\right). \tag{1}$$

The expectation $\mathbb{E}$ is taken over an ensemble of *random frequencies* $\{\boldsymbol{\omega}\}_{i=1}^m$ drawn from a distribution η. The space in which $\{\boldsymbol{\omega}\}_{i=1}^m$ live and manner in which they are combined to construct $\phi(\boldsymbol{x})$ depends on the particular input space $\mathcal{X}$ and kernel function k being approximated. This paper will consider several examples. The set of RFs $\{\phi(\boldsymbol{x}_i)\}_{i=1}^N$ can be used to construct a low-rank or sparse approximation of the Gram matrix, providing substantial space and time complexity savings. RFs exist for a variety of kernels, including for continuous and discrete input spaces (Dasgupta et al., 2010; Johnson, 1984; Choromanski et al., 2020; Goemans and Williamson, 2001; Rahimi and Recht, 2007; Choromanski, 2023; Tripp et al., 2024).

Variance reduction for RFs. Eq. 1 can be understood as a *Monte Carlo* (MC) estimate of k. In applications, it is often found that this estimate converges slowly. This can be addressed by taking many samples m, but this undermines the efficiency gains of RFs. Therefore, substantial effort has been dedicated to *reducing the variance* of the kernel estimates. Variance reduction methods include quasi-Monte Carlo (QMC; Dick et al., 2013; Yang et al., 2014a), common random numbers (CRNs; Glasserman and Yao, 1992), antithetic variates (Hammersley and Morton, 1956) and structured Monte Carlo (SMC; Yu et al., 2016). These techniques work by replacing i.i.d. frequencies $\{\boldsymbol{\omega}_i\}_{i=1}^m$ by a *dependent ensemble*, with the sample dependencies designed to improve RF convergence.

Journal of Machine Learning **ISSN**: 2790-2048(e), 2790-203X(p)

Bridging Traditional and Machine Learning-based Algorithms for Solving PDEs: The Random Feature Method

Jingrun Chen * [1,2], Xurong Chi † [1], Weinan E ‡ [3,4], and Zhouwang Yang § [1,4]

[1]School of Mathematical Sciences, University of Science and Technology of China
[2]Suzhou Institute for Advanced Research, University of Science and Technology of China
[3]AI for Science Institute, Beijing and Center for Machine Learning Research and School of Mathematical Sciences, Peking University
[4]School of Data Science, University of Science and Technology of China

Abstract. One of the oldest and most studied subject in scientific computing is algorithms for solving partial differential equations (PDEs). A long list of numerical methods have been proposed and successfully used for various applications. In recent years, deep learning methods have shown their superiority for high-dimensional PDEs where traditional methods fail. However, for low dimensional problems, it remains unclear whether these methods have a real advantage over traditional algorithms as a direct solver. In this work, we propose the random feature method (RFM) for solving PDEs, a natural bridge between traditional and machine learning-based algorithms. RFM is based on a combination of well-known ideas: 1. representation of the approximate solution using random feature functions; 2. collocation method to take care of the PDE; 3. the penalty method to treat the boundary conditions, which allows us to treat the boundary condition and the PDE in the same footing. We find it crucial to add several additional components including multi-scale representation and rescaling the weights in the loss function. We demonstrate that the method exhibits spectral accuracy and can compete with traditional solvers in terms of both accuracy and efficiency. In addition, we find that RFM is particularly suited for complex problems with complex geometry, where both traditional and machine learning-based algorithms encounter difficulties.

Keywords:
Partial differential equation,
Machine learning,
Random feature method,
Rescaling

Article Info.:
Volume: 1
Number: 2
Pages: 1- xx
Date: June/2022
doi.org/10.4208/jml.xxx

Article History:
Received: xx/xx/2022
Accepted: xx/xx/2022

Communicated by:
xxx

1 Introduction

One of the oldest and most studied subject in scientific computing is algorithms for solving partial differential equations (PDEs). Finite difference [13], finite element [25], spectral methods [20] and a host of other methodologies have been proposed and studied, with great success. At the same time, a variety of scientific softwares based on these methodologies have been developed and widely used by the academia as well as the industry. They have become standard resources in most, if not all, engineering applications.

*jingrunchen@ustc.edu.cn.
†cxr123@mail.ustc.edu.cn.
‡weinan@math.pku.edu.cn.
§yangzw@ustc.edu.cn.